

Day 3：前端项目架构初始化 + 公共组件 — 教程

目录

- [一、今日目标与验收标准](#)
 - [二、前端目录结构设计与模块划分](#)
 - [三、安装核心依赖](#)
 - [四、Vite 配置与环境变量](#)
 - [五、全局样式与 Element Plus 主题配置](#)
 - [六、Axios 请求封装](#)
 - [七、SSE 流式处理工具封装](#)
 - [八、Pinia 状态管理](#)
 - [九、Vue Router 路由配置与路由守卫](#)
 - [十、布局组件 \(AppHeader / AppSidebar / MainLayout\)](#)
 - [十一、登录页与注册页实现](#)
 - [十二、Markdown 渲染工具封装](#)
 - [十三、入口文件整合](#)
 - [十四、验证与测试](#)
 - [十五、常见问题排查](#)
-

一、今日目标与验收标准

1.1 今日任务总览

时间段	内容	形式
上午 0.5h	Vue 3 项目初始化（Vite）、目录结构讲解	实操
上午 0.5h	Element Plus 集成、全局样式、主题配置	实操
上午 0.5h	Vue Router 路由配置（登录/注册/对话/检测/训练/历史）	实操
上午 0.5h	Pinia 状态管理：user store、agent store	实操
上午 0.5h	Axios 封装：请求拦截器、Token 注入、错误处理	实操
上午 0.5h	SSE 流式处理工具封装	实操
下午 1h	AppHeader + AppSidebar 布局组件	实操
下午 1h	登录页 + 注册页实现（对接后端认证 API）	实操
下午 0.5h	页面路由守卫：未登录跳转登录页	实操
下午 0.5h	Markdown 渲染工具（markdown-it 封装）	实操

1.2 验收标准

- ☐ npm run dev 前端可启动，访问 http://localhost:5173
- ☐ 登录/注册页面可正常提交并获取 Token
- ☐ 登录后跳转到主界面，侧边栏导航可切换
- ☐ Axios 拦截器正常工作，Token 自动注入
- ☐ 未登录访问受保护页面时自动跳转到登录页

二、前端目录结构与模块划分

2.1 目标目录结构

Day 3 结束后，frontend 目录应如下：

```
frontend/
├── index.html           # HTML 入口
├── vite.config.js       # Vite 配置
├── package.json         # 项目依赖
├── .env                 # 环境变量
├── .env.example         # 环境变量模板
├──
├── public/
│   └── favicon.svg      # 网站图标
├──
└── src/
    ├── main.js          # 应用入口
    └── App.vue           # 根组件
```

└─ assets/	# 静态资源
└─ styles/	
└─ variables.scss	# 全局 SCSS 变量
└─ reset.scss	# 样式重置
└─ global.scss	# 全局样式
└─ utils/	# 工具函数
└─ request.js	# Axios 封装
└─ stream.js	# SSE 流式处理
└─ markdown.js	# Markdown 渲染
└─ stores/	# Pinia 状态管理
└─ index.js	# Pinia 实例
└─ user.js	# 用户状态
└─ agent.js	# 智能体状态
└─ router/	# 路由配置
└─ index.js	# 路由定义 + 守卫
└─ api/	# API 接口层
└─ auth.js	# 认证相关 API
└─ views/	# 页面组件
└─ LoginPage.vue	# 登录页
└─ RegisterPage.vue	# 注册页
└─ ChatPage.vue	# 智能对话页 (Day 11 完善)
└─ DetectionPage.vue	# 检测工作台 (Day 8 完善)
└─ TrainingPage.vue	# 模型训练页 (Day 6 完善)
└─ HistoryPage.vue	# 历史记录页 (Day 10 完善)
└─ DashboardPage.vue	# 数据看板 (Day 10 完善)
└─ components/	# 公共组件
└─ layout/	
└─ AppHeader.vue	# 顶部导航栏
└─ AppSidebar.vue	# 侧边栏菜单
└─ MainLayout.vue	# 主布局 (Header + Sidebar + Content)
└─ App.vue	# 根组件

2.2 模块划分原则

目录	职责	说明
assets/styles/	全局样式	SCSS 变量、样式重置、全局样式
utils/	工具函数	Axios 封装、SSE 流式、Markdown 渲染
stores/	状态管理	Pinia store，管理用户信息、智能体对话
router/	路由配置	页面路由定义、导航守卫
api/	接口层	封装所有后端 API 调用
views/	页面组件	每个路由对应一个页面
components/	公共组件	布局组件、可复用 UI 组件

三、安装核心依赖

3.1 一次性安装所有依赖

```
cd frontend

# UI 框架 + 图标
npm install element-plus @element-plus/icons-vue

# 路由 + 状态管理
npm install vue-router@4 pinia

# HTTP 请求
npm install axios

# Markdown 渲染
npm install markdown-it

# SCSS 支持 (vite 内置 sass 支持, 只需安装 sass)
npm install -D sass
```

3.2 依赖说明

依赖	版本	用途
<code>element-plus</code>	latest	UI 组件库（表格、表单、对话框等）
<code>@element-plus/icons-vue</code>	latest	Element Plus 图标集
<code>vue-router@4</code>	4.x	Vue 3 路由管理
<code>pinia</code>	latest	Vue 3 状态管理（替代 Vuex）
<code>axios</code>	latest	HTTP 请求库
<code>markdown-it</code>	latest	Markdown 渲染（Day 11 智能体对话使用）
<code>sass</code>	latest	SCSS 预处理器

四、Vite 配置与环境变量

4.1 环境变量文件

创建 `frontend/.env`：

```
# =====
# 前端环境变量配置
# =====

# 后端 API 地址 (Vite 开发服务器代理目标)
VITE_API_BASE_URL=http://localhost:8000

# 应用名称
VITE_APP_TITLE=RSOD Agent Platform

# MinIO 文件访问地址
VITE_MINIO_URL=http://localhost:9000
```

创建 `frontend/.env.example`（内容与 `.env` 相同，供团队参考）：

```
VITE_API_BASE_URL=http://localhost:8000
VITE_APP_TITLE=RSOD Agent Platform
VITE_MINIO_URL=http://localhost:9000
```

4.2 Vite 配置

修改 `frontend/vite.config.js`：

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import path from 'path'
```

```
export default defineConfig({
  plugins: [vue()],

  // — 路径别名 —————
  resolve: {
    alias: {
      '@': path.resolve(__dirname, 'src'),
    },
  },

  // — CSS 预处理器配置 —————
  css: {
    preprocessorOptions: {
      scss: {
        additionalData: `@use "@/assets/styles/variables.scss" as *;`,
      },
    },
  },

  // — 开发服务器配置 —————
  server: {
    port: 5173,
    open: true, // 启动时自动打开浏览器

    // API 代理: 将 /api 开头的请求转发到后端
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
      },
    },
  },
})
```

配置说明:

配置项	说明
<code>resolve.alias</code>	@ 指向 <code>src/</code> 目录，方便引用: <code>import request from '@/utils/request'</code>
<code>css.preprocessorOptions</code>	全局注入 SCSS 变量文件，所有组件可直接使用变量
<code>server.proxy</code>	开发环境将 <code>/api</code> 请求代理到后端 <code>localhost:8000</code> ，避免跨域问题

五、全局样式与 Element Plus 主题配置

5.1 SCSS 变量文件

创建 `src/assets/styles/variables.scss` :

```
// =====
// 全局 SCSS 变量 — 主题色、间距、尺寸
// =====

// — 主题色 —
$primary-color: #409eff;
$success-color: #67c23a;
$warning-color: #e6a23c;
$danger-color: #f56c6c;
$info-color: #909399;

// — 背景色 —
$bg-color: #f5f7fa;
$bg-color-dark: #1d1e1f;
$sidebar-bg: #304156;
$sidebar-text: #bfc9d9;
$sidebar-active-text: #409eff;

// — 文字色 —
$text-primary: #303133;
$text-regular: #606266;
$text-secondary: #909399;
$text-placeholder: #c0c4cc;

// — 布局尺寸 —
$header-height: 56px;
$sidebar-width: 210px;
$sidebar-collapsed-width: 64px;

// — 间距 —
$spacing-xs: 4px;
$spacing-sm: 8px;
$spacing-md: 16px;
$spacing-lg: 24px;
$spacing-xl: 32px;

// — 圆角 —
$border-radius-sm: 4px;
$border-radius-md: 8px;
$border-radius-lg: 12px;

// — 阴影 —
$shadow-sm: 0 1px 3px rgba(0, 0, 0, 0.08);
$shadow-md: 0 2px 8px rgba(0, 0, 0, 0.1);
$shadow-lg: 0 4px 16px rgba(0, 0, 0, 0.12);
```

5.2 样式重置

创建 `src/assets/styles/reset.scss`：

```
// =====
```

```
// CSS 样式重置
// =====

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html, body {
  width: 100%;
  height: 100%;
  font-family: 'Helvetica Neue', Helvetica, 'PingFang SC', 'Hiragino Sans GB',
    'Microsoft YaHei', Arial, sans-serif;
  font-size: 14px;
  color: #303133;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

#app {
  width: 100%;
  height: 100%;
}

a {
  color: inherit;
  text-decoration: none;
}

ul, ol {
  list-style: none;
}

img {
  max-width: 100%;
  vertical-align: middle;
}

input, button, textarea {
  font-family: inherit;
  font-size: inherit;
}
```

5.3 全局样式

创建 `src/assets/styles/global.scss`：

```
// =====
// 全局样式 — 通用类名、布局辅助
// =====
```

```
@use './reset.scss';
```

```
// — 页面容器
```

```
.page-container {  
  padding: $spacing-lg;  
  min-height: calc(100vh - $header-height);  
}
```

```
// — 卡片容器
```

```
.card-container {  
  background: #fff;  
  border-radius: $border-radius-md;  
  box-shadow: $shadow-sm;  
  padding: $spacing-lg;  
  margin-bottom: $spacing-md;  
}
```

```
// — Flex 布局辅助
```

```
.flex-center {  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

```
.flex-between {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
}
```

```
// — 文本截断
```

```
.text-ellipsis {  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
}
```

```
// — 滚动条美化
```

```
::-webkit-scrollbar {  
  width: 6px;  
  height: 6px;  
}
```

```
::-webkit-scrollbar-thumb {  
  background: #c0c4cc;  
  border-radius: 3px;  
}
```

```
::-webkit-scrollbar-track {  
  background: transparent;  
}
```

六、Axios 请求封装

6.1 创建请求实例

创建 `src/utils/request.js`：

```
/**
 * Axios 请求封装
 * - 统一 baseURL 配置
 * - 请求拦截器：自动注入 JWT Token
 * - 响应拦截器：统一错误处理、Token 过期处理
 */
import axios from 'axios'
import { ElMessage } from 'element-plus'
import { useUserStore } from '@/stores/user'
import router from '@/router'

// — 创建 Axios 实例 —————
const request = axios.create({
  baseURL: '/api',          // 配合 Vite proxy, 实际请求转发到后端
  timeout: 30000,           // 请求超时 30 秒
  headers: {
    'Content-Type': 'application/json',
  },
})

// — 请求拦截器 —————
request.interceptors.request.use(
  (config) => {
    // 从 Pinia store 获取 Token, 自动注入请求头
    const userStore = useUserStore()
    if (userStore.token) {
      config.headers.Authorization = `Bearer ${userStore.token}`
    }
    return config
  },
  (error) => {
    return Promise.reject(error)
  }
)

// — 响应拦截器 —————
request.interceptors.response.use(
  (response) => {
    // 请求成功, 直接返回响应数据
    return response.data
  },
  (error) => {
    const { response } = error
```

```

if (response) {
  switch (response.status) {
    case 401:
      // Token 过期或无效, 清除用户信息并跳转登录页
      ElMessage.error('登录已过期, 请重新登录')
      const userStore = useUserStore()
      userStore.logout()
      router.push('/login')
      break

    case 403:
      ElMessage.error('没有权限执行此操作')
      break

    case 404:
      ElMessage.error('请求的资源不存在')
      break

    case 422:
      // Pydantic 验证错误
      const detail = response.data?.detail
      if (Array.isArray(detail)) {
        ElMessage.error(detail[0]?.msg || '参数验证失败')
      } else {
        ElMessage.error(detail || '参数验证失败')
      }
      break

    case 500:
      ElMessage.error('服务器内部错误')
      break

    default:
      ElMessage.error(response.data?.detail || `请求失败 (${response.status})`)
  }
} else {
  // 网络错误或请求超时
  ElMessage.error('网络连接异常, 请检查后端服务是否启动')
}

return Promise.reject(error)
}
)

export default request

```

6.2 认证 API 接口

创建 `src/api/auth.js`:

```
/**
```

```

* 认证相关 API 接口
*/
import request from '@utils/request'

/**
 * 用户注册
 * @param {Object} data - { username, email, password }
 */
export function registerApi(data) {
  return request.post('/auth/register', data)
}

/**
 * 用户登录
 * @param {Object} data - { username, password }
 * @returns {Promise} - { access_token, token_type, user }
 */
export function loginApi(data) {
  return request.post('/auth/login', data)
}

/**
 * 获取当前用户信息 (需要 Token)
 */
export function getUserInfoApi() {
  return request.get('/auth/me')
}

```

七、SSE 流式处理工具封装

7.1 SSE 工具函数

创建 `src/utils/stream.js`：

```

/**
 * SSE (Server-Sent Events) 流式处理工具
 * 用于 Day 11 智能体对话的流式渲染
 *
 * 使用方式:
 *   const stop = streamChat(
 *     '/api/chat/stream',
 *     { message: '你好' },
 *     {
 *       onMessage: (chunk) => { content += chunk },
 *       onDone: () => { console.log('完成') },
 *       onError: (err) => { console.error(err) },
 *     }
 *   )
 */

```

```

/**
 * 发起 SSE 流式请求
 *
 * @param {string} url - 请求地址（相对路径，会经过 Vite proxy）
 * @param {Object} body - 请求体
 * @param {Object} callbacks - 回调函数
 * @param {Function} callbacks.onMessage - 收到消息片段时的回调
 * @param {Function} callbacks.onDone - 流结束时的回调
 * @param {Function} callbacks.onError - 错误时的回调
 * @returns {Function} stop - 调用此函数可中断连接
 */
export function streamChat(url, body, callbacks) {
  const { onMessage, onDone, onError } = callbacks

  // 从 localStorage 获取 Token
  const token = localStorage.getItem('rsod_token')

  // 使用 fetch + ReadableStream 实现 SSE
  const controller = new AbortController()

  fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: JSON.stringify(body),
    signal: controller.signal,
  })
  .then(async (response) => {
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`)
    }

    const reader = response.body.getReader()
    const decoder = new TextDecoder('utf-8')

    while (true) {
      const { done, value } = await reader.read()
      if (done) {
        onDone?.()
        break
      }

      // 解析 SSE 数据
      const text = decoder.decode(value, { stream: true })
      const lines = text.split('\n')

      for (const line of lines) {
        if (line.startsWith('data: ')) {
          const data = line.slice(6) // 去掉 "data: " 前缀

```

```

        if (data === '[DONE]') {
          onDone?.()
          return
        }

        try {
          const parsed = JSON.parse(data)
          onMessage?.(parsed)
        } catch {
          // 非 JSON 数据，直接作为文本片段
          onMessage?.(data)
        }
      }
    }
  })
})
.catch((err) => {
  if (err.name !== 'AbortError') {
    onError?.(err)
  }
})
})

// 返回中断函数
return () => controller.abort()
}

```

八、Pinia 状态管理

8.1 Pinia 实例

创建 `src/stores/index.js`：

```

/**
 * Pinia 状态管理入口
 */
import { createPinia } from 'pinia'

const pinia = createPinia()

export default pinia

```

8.2 用户状态

创建 `src/stores/user.js`：

```

/**
 * 用户状态管理
 * 管理用户登录信息、Token、角色等
 */

```

```

import { defineStore } from 'pinia'
import { loginApi, getUserInfoApi } from '@/api/auth'

const TOKEN_KEY = 'rsod_token'
const USER_KEY = 'rsod_user'

export const useUserStore = defineStore('user', {
  state: () => ({
    // JWT Token
    token: localStorage.getItem(TOKEN_KEY) || '',
    // 用户信息
    user: JSON.parse(localStorage.getItem(USER_KEY) || 'null'),
  }),

  getters: {
    /** 是否已登录 */
    isLoggedIn: (state) => !!state.token,

    /** 用户名 */
    username: (state) => state.user?.username || '',

    /** 用户头像 */
    avatar: (state) => state.user?.avatar || '',

    /** 用户角色列表 */
    roles: (state) => state.user?.roles || [],

    /** 是否为管理员 */
    isSuperuser: (state) => state.user?.is_superuser || false,
  },

  actions: {
    /**
     * 用户登录
     * @param {Object} credentials - { username, password }
     */
    async login(credentials) {
      const res = await loginApi(credentials)

      // 保存 Token
      this.token = res.access_token
      localStorage.setItem(TOKEN_KEY, res.access_token)

      // 保存用户信息
      this.user = res.user
      localStorage.setItem(USER_KEY, JSON.stringify(res.user))

      return res
    },

    /**
     * 获取最新用户信息

```

```

    */
    async fetchUserInfo() {
      try {
        const user = await getUserInfoApi()
        this.user = user
        localStorage.setItem(USER_KEY, JSON.stringify(user))
      } catch {
        this.logout()
      }
    },

    /**
     * 退出登录
     */
    logout() {
      this.token = ''
      this.user = null
      localStorage.removeItem(TOKEN_KEY)
      localStorage.removeItem(USER_KEY)
    },
  },
})

```

8.3 智能体状态

创建 `src/stores/agent.js`：

```

/**
 * 智能体对话状态管理
 * 管理对话会话列表、当前会话消息等
 */
import { defineStore } from 'pinia'

export const useAgentStore = defineStore('agent', {
  state: () => ({
    // 当前会话 ID
    currentSessionId: null,

    // 当前会话的消息列表
    messages: [],

    // 会话列表
    sessions: [],

    // 是否正在等待 AI 响应
    isLoading: false,

    // 中断函数（用于取消 SSE 流式请求）
    abortController: null,
  }),
})

```

```

getters: {
  /** 消息数量 */
  messageCount: (state) => state.messages.length,

  /** 是否有会话 */
  hasSession: (state) => state.sessions.length > 0,
},

actions: {
  /** 添加一条消息 */
  addMessage(message) {
    this.messages.push(message)
  },

  /** 更新最后一条 AI 消息（用于流式追加） */
  updateLastAssistantMessage(content) {
    const lastMsg = this.messages[this.messages.length - 1]
    if (lastMsg && lastMsg.role === 'assistant') {
      lastMsg.content = content
    }
  },

  /** 设置加载状态 */
  setLoading/loading) {
    this.isLoading = loading
  },

  /** 中断当前流式请求 */
  abort() {
    if (this.abortController) {
      this.abortController()
      this.abortController = null
      this.isLoading = false
    }
  },

  /** 新建对话 */
  newChat() {
    this.currentSessionId = null
    this.messages = []
    this.abort()
  },

  /** 清除所有状态 */
  clear() {
    this.currentSessionId = null
    this.messages = []
    this.sessions = []
    this.abort()
  },
},
})

```

九、Vue Router 路由配置与路由守卫

9.1 路由配置

创建 `src/router/index.js` :

```
/**
 * Vue Router 路由配置
 * - 登录/注册页面无需认证
 * - 其他页面需要登录后才能访问
 * - 路由守卫自动检查登录状态
 */
import { createRouter, createWebHistory } from 'vue-router'

// — 路由定义 —————
const routes = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@/views/LoginPage.vue'),
    meta: { title: '登录', requiresAuth: false },
  },
  {
    path: '/register',
    name: 'Register',
    component: () => import('@/views/RegisterPage.vue'),
    meta: { title: '注册', requiresAuth: false },
  },
  // — 需要登录的页面（使用 MainLayout 布局） —————
  {
    path: '/',
    component: () => import('@/components/layout/MainLayout.vue'),
    redirect: '/chat',
    meta: { requiresAuth: true },
    children: [
      {
        path: 'chat',
        name: 'Chat',
        component: () => import('@/views/ChatPage.vue'),
        meta: { title: '智能对话', icon: 'ChatDotRound' },
      },
      {
        path: 'detection',
        name: 'Detection',
        component: () => import('@/views/DetectionPage.vue'),
        meta: { title: '检测工作台', icon: 'Camera' },
      },
      {
        path: 'training',
```

```

    name: 'Training',
    component: () => import('@views/TrainingPage.vue'),
    meta: { title: '模型训练', icon: 'Cpu' },
  },
  {
    path: 'history',
    name: 'History',
    component: () => import('@views/HistoryPage.vue'),
    meta: { title: '历史记录', icon: 'Clock' },
  },
  {
    path: 'dashboard',
    name: 'Dashboard',
    component: () => import('@views/DashboardPage.vue'),
    meta: { title: '数据看板', icon: 'DataAnalysis' },
  },
],
},
],

// — 404 页面 —————
{
  path: '/*:pathMatch(.*)*',
  redirect: '/login',
},
]

// — 创建路由实例 —————
const router = createRouter({
  history: createWebHistory(),
  routes,
})

// — 路由守卫 —————
router.beforeEach((to, from, next) => {
  // 设置页面标题
  document.title = to.meta.title
    ? `${to.meta.title} - RSOD Agent Platform`
    : 'RSOD Agent Platform'

  // 检查是否需要认证
  const token = localStorage.getItem('rsod_token')
  const requiresAuth = to.matched.some((record) => record.meta.requiresAuth !== false)

  if (requiresAuth && !token) {
    // 需要登录但未登录, 跳转到登录页
    next({ path: '/login', query: { redirect: to.fullPath } })
  } else if ((to.path === '/login' || to.path === '/register') && token) {
    // 已登录用户访问登录/注册页, 跳转到首页
    next('/')
  } else {
    next()
  }
})

```

```
  })

  export default router
```

路由设计说明：

设计点	说明
<code>meta.requiresAuth</code>	标记路由是否需要认证，守卫中统一检查
<code>meta.title</code>	页面标题，守卫中自动设置到 <code>document.title</code>
<code>meta.icon</code>	侧边栏菜单图标名称（对应 Element Plus Icons）
懒加载 <code>() => import(...)</code>	所有页面组件按需加载，减少首屏加载时间
嵌套路由 <code>children</code>	需要登录的页面共享 <code>MainLayout</code> 布局

十、布局组件（AppHeader / AppSidebar / MainLayout）

10.1 顶部导航栏

创建 `src/components/layout/AppHeader.vue`：

```
<template>
  <header class="app-header">
    <!-- 左侧：Logo + 平台名称 -->
    <div class="header-left">
      
      <span class="header-title">RSOD Agent Platform</span>
    </div>

    <!-- 右侧：用户信息 + 退出按钮 -->
    <div class="header-right">
      <el-dropdown trigger="click" @command="handleCommand">
        <div class="user-info">
          <el-avatar :size="32" :src="userStore.avatar || undefined">
            {{ userStore.username?.charAt(0)?.toUpperCase() }}
          </el-avatar>
          <span class="username">{{ userStore.username }}</span>
          <el-icon><ArrowDown /></el-icon>
        </div>
        <template #dropdown>
          <el-dropdown-menu>
            <el-dropdown-item command="profile">
              <el-icon><User /></el-icon>个人中心
            </el-dropdown-item>
            <el-dropdown-item command="logout" divided>
              <el-icon><SwitchButton /></el-icon>退出登录
            </el-dropdown-item>
          </el-dropdown-menu>
        </template>
      </el-dropdown>
    </div>
  </header>
</template>
```

```

        </el-dropdown-menu>
      </template>
    </el-dropdown>
  </div>
</header>
</template>

<script setup>
import { useRouter } from 'vue-router'
import { ArrowDown, User, SwitchButton } from '@element-plus/icons-vue'
import { ElMessageBox } from 'element-plus'
import { useUserStore } from '@/stores/user'

const router = useRouter()
const userStore = useUserStore()

/** 处理下拉菜单命令 */
function handleCommand(command) {
  switch (command) {
    case 'profile':
      // 个人中心 (后续实现)
      break
    case 'logout':
      ElMessageBox.confirm('确定要退出登录吗?', '提示', {
        confirmButtonText: '确定',
        cancelButtonText: '取消',
        type: 'warning',
      }).then(() => {
        userStore.logout()
        router.push('/login')
      }).catch(() => {})
      break
  }
}
</script>

<style lang="scss" scoped>
.app-header {
  height: $header-height;
  background: #fff;
  border-bottom: 1px solid #ebeef5;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 $spacing-lg;
  box-shadow: $shadow-sm;
  z-index: 100;
}

.header-left {
  display: flex;
  align-items: center;

```

```

    gap: $spacing-sm;
  }

.header-logo {
  width: 28px;
  height: 28px;
}

.header-title {
  font-size: 16px;
  font-weight: 600;
  color: $text-primary;
}

.header-right {
  display: flex;
  align-items: center;
}

.user-info {
  display: flex;
  align-items: center;
  gap: $spacing-sm;
  cursor: pointer;
  padding: 4px 8px;
  border-radius: $border-radius-sm;
  transition: background 0.2s;

  &:hover {
    background: #f5f7fa;
  }
}

.username {
  font-size: 14px;
  color: $text-primary;
}
</style>

```

10.2 侧边栏菜单

创建 `src/components/layout/AppSidebar.vue` :

```

<template>
  <aside class="app-sidebar">
    <el-menu
      :default-active="activeMenu"
      :router="true"
      background-color="#304156"
      text-color="#bfcbd9"
      active-text-color="#409eff"
    >

```

```

    >
    <el-menu-item
      v-for="item in menuItems"
      :key="item.path"
      :index="item.path"
    >
      <el-icon>
        <component :is="item.icon" />
      </el-icon>
      <span>{{ item.title }}</span>
    </el-menu-item>
  </el-menu>
</aside>
</template>

<script setup>
import { computed } from 'vue'
import { useRoute } from 'vue-router'
import {
  ChatDotRound,
  Camera,
  Cpu,
  Clock,
  DataAnalysis,
} from '@element-plus/icons-vue'

const route = useRoute()

/** 当前激活的菜单项 */
const activeMenu = computed(() => {
  return '/' + route.path.split('/')[1]
})

/** 侧边栏菜单配置 */
const menuItems = [
  { path: '/chat', title: '智能对话', icon: ChatDotRound },
  { path: '/detection', title: '检测工作台', icon: Camera },
  { path: '/training', title: '模型训练', icon: Cpu },
  { path: '/history', title: '历史记录', icon: Clock },
  { path: '/dashboard', title: '数据看板', icon: DataAnalysis },
]
</script>

<style lang="scss" scoped>
.app-sidebar {
  width: $sidebar-width;
  height: 100%;
  background: $sidebar-bg;
  overflow-y: auto;

  .el-menu {
    border-right: none;
  }
}

```

```

    height: 100%;
  }

  .el-menu-item {
    height: 50px;
    line-height: 50px;

    &.is-active {
      background-color: rgba(64, 158, 255, 0.15) !important;
    }

    &:hover {
      background-color: rgba(255, 255, 255, 0.05) !important;
    }
  }
}
</style>

```

10.3 主布局

创建 `src/components/layout/MainLayout.vue`：

```

<template>
  <div class="main-layout">
    <!-- 顶部导航栏 -->
    <AppHeader />

    <!-- 下方区域：侧边栏 + 内容区 -->
    <div class="layout-body">
      <AppSidebar />

      <!-- 页面内容区 -->
      <main class="layout-content">
        <router-view />
      </main>
    </div>
  </div>
</template>

<script setup>
import AppHeader from './AppHeader.vue'
import AppSidebar from './AppSidebar.vue'
</script>

<style lang="scss" scoped>
.main-layout {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
}

```

```
.layout-body {  
  flex: 1;  
  display: flex;  
  overflow: hidden;  
}  
  
.layout-content {  
  flex: 1;  
  background: $bg-color;  
  overflow-y: auto;  
  padding: $spacing-lg;  
}  
</style>
```

十一、登录页与注册页实现

11.1 登录页

创建 `src/views/LoginPage.vue` :

```
<template>  
  <div class="login-page">  
    <div class="login-card">  
      <div class="login-header">  
          
        <h2>RSOD Agent Platform</h2>  
        <p>基于 YOLOv11 的目标检测智能体平台</p>  
      </div>  
  
      <el-form  
        ref="formRef"  
        :model="loginForm"  
        :rules="loginRules"  
        label-width="0"  
        size="large"  
        @submit.prevent="handleLogin"  
      >  
        <el-form-item prop="username">  
          <el-input  
            v-model="loginForm.username"  
            placeholder="请输入用户名"  
            prefix-icon="User"  
          />  
        </el-form-item>  
  
        <el-form-item prop="password">  
          <el-input  
            v-model="loginForm.password"  
            type="password"  
          />  
        </el-form-item>  
      </el-form>  
    </div>  
  </div>  
</template>
```

```

        placeholder="请输入密码"
        prefix-icon="Lock"
        show-password
        @keyup.enter="handleLogin"
      />
    </el-form-item>

    <el-form-item>
      <el-button
        type="primary"
        class="login-btn"
        :loading="loading"
        @click="handleLogin"
      >
        登 录
      </el-button>
    </el-form-item>
  </el-form>

  <div class="login-footer">
    <span>还没有账号? </span>
    <router-link to="/register">立即注册</router-link>
  </div>
</div>
</div>
</template>

<script setup>
import { ref, reactive } from 'vue'
import { useRouter, useRoute } from 'vue-router'
import { ElMessage } from 'element-plus'
import { useUserStore } from '@/stores/user'

const router = useRouter()
const route = useRoute()
const userStore = useUserStore()

const formRef = ref(null)
const loading = ref(false)

/** 登录表单数据 */
const loginForm = reactive({
  username: '',
  password: '',
})

/** 表单验证规则 */
const loginRules = {
  username: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 3, max: 50, message: '用户名长度为 3-50 个字符', trigger: 'blur' },
  ],

```

```

password: [
  { required: true, message: '请输入密码', trigger: 'blur' },
  { min: 6, message: '密码至少 6 个字符', trigger: 'blur' },
],
},
}

/** 处理登录 */
async function handleLogin() {
  const valid = await formRef.value.validate().catch(() => false)
  if (!valid) return

  loading.value = true
  try {
    await userStore.login({
      username: loginForm.username,
      password: loginForm.password,
    })

    ElMessage.success('登录成功')

    // 跳转到目标页面（如果有 redirect 参数）或首页
    const redirect = route.query.redirect || '/'
    router.push(redirect)
  } catch {
    // 错误已在 Axios 拦截器中统一处理
  } finally {
    loading.value = false
  }
}
</script>

<style lang="scss" scoped>
.login-page {
  width: 100%;
  height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.login-card {
  width: 420px;
  padding: 40px;
  background: #fff;
  border-radius: $border-radius-lg;
  box-shadow: $shadow-lg;
}

.login-header {
  text-align: center;
  margin-bottom: 32px;
}

```

```

.login-logo {
  width: 48px;
  height: 48px;
  margin-bottom: 12px;
}

h2 {
  font-size: 22px;
  color: $text-primary;
  margin-bottom: 8px;
}

p {
  font-size: 13px;
  color: $text-secondary;
}

}

.login-btn {
  width: 100%;
}

.login-footer {
  text-align: center;
  font-size: 13px;
  color: $text-secondary;

  a {
    color: $primary-color;
    margin-left: 4px;

    &:hover {
      text-decoration: underline;
    }
  }
}
}
</style>

```

11.2 注册页

创建 `src/views/RegisterPage.vue` :

```

<template>
  <div class="register-page">
    <div class="register-card">
      <div class="register-header">
        
        <h2>创建账号</h2>
        <p>注册后即可使用目标检测智能体平台</p>
      </div>
    </div>
  </div>
</template>

```

```
<el-form
  ref="formRef"
  :model="registerForm"
  :rules="registerRules"
  label-width="0"
  size="large"
  @submit.prevent="handleRegister"
>
  <el-form-item prop="username">
    <el-input
      v-model="registerForm.username"
      placeholder="请输入用户名"
      prefix-icon="User"
    />
  </el-form-item>

  <el-form-item prop="email">
    <el-input
      v-model="registerForm.email"
      placeholder="请输入邮箱"
      prefix-icon="Message"
    />
  </el-form-item>

  <el-form-item prop="password">
    <el-input
      v-model="registerForm.password"
      type="password"
      placeholder="请输入密码（至少 6 位）"
      prefix-icon="Lock"
      show-password
    />
  </el-form-item>

  <el-form-item prop="confirmPassword">
    <el-input
      v-model="registerForm.confirmPassword"
      type="password"
      placeholder="请确认密码"
      prefix-icon="Lock"
      show-password
      @keyup.enter="handleRegister"
    />
  </el-form-item>

  <el-form-item>
    <el-button
      type="primary"
      class="register-btn"
      :loading="loading"
      @click="handleRegister"
    />
  </el-form-item>
</el-form>
```

```

    >
    注册
  </el-button>
</el-form-item>
</el-form>

<div class="register-footer">
  <span>已有账号? </span>
  <router-link to="/login">立即登录</router-link>
</div>
</div>
</div>
</template>

<script setup>
import { ref, reactive } from 'vue'
import { useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
import { registerApi } from '@/api/auth'

const router = useRouter()
const formRef = ref(null)
const loading = ref(false)

/** 注册表单数据 */
const registerForm = reactive({
  username: '',
  email: '',
  password: '',
  confirmPassword: '',
})

/** 确认密码验证器 */
const validateConfirmPassword = (rule, value, callback) => {
  if (value !== registerForm.password) {
    callback(new Error('两次输入的密码不一致'))
  } else {
    callback()
  }
}

/** 表单验证规则 */
const registerRules = {
  username: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 3, max: 50, message: '用户名长度为 3-50 个字符', trigger: 'blur' },
  ],
  email: [
    { required: true, message: '请输入邮箱', trigger: 'blur' },
    { type: 'email', message: '请输入有效的邮箱地址', trigger: 'blur' },
  ],
  password: [

```

```

    { required: true, message: '请输入密码', trigger: 'blur' },
    { min: 6, message: '密码至少 6 个字符', trigger: 'blur' },
  ],
  confirmPassword: [
    { required: true, message: '请确认密码', trigger: 'blur' },
    { validator: validateConfirmPassword, trigger: 'blur' },
  ],
}

```

/ 处理注册 */**

```

async function handleRegister() {
  const valid = await formRef.value.validate().catch(() => false)
  if (!valid) return

  loading.value = true
  try {
    await registerApi({
      username: registerForm.username,
      email: registerForm.email,
      password: registerForm.password,
    })

    ElMessage.success('注册成功, 请登录')
    router.push('/login')
  } catch {
    // 错误已在 Axios 拦截器中统一处理
  } finally {
    loading.value = false
  }
}
</script>

```

<style lang="scss" scoped>

```

.register-page {
  width: 100%;
  height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

```

```

.register-card {
  width: 420px;
  padding: 40px;
  background: #fff;
  border-radius: $border-radius-lg;
  box-shadow: $shadow-lg;
}

```

```

.register-header {
  text-align: center;
}

```

```
margin-bottom: 32px;

.register-logo {
  width: 48px;
  height: 48px;
  margin-bottom: 12px;
}

h2 {
  font-size: 22px;
  color: $text-primary;
  margin-bottom: 8px;
}

p {
  font-size: 13px;
  color: $text-secondary;
}

.register-btn {
  width: 100%;
}

.register-footer {
  text-align: center;
  font-size: 13px;
  color: $text-secondary;

  a {
    color: $primary-color;
    margin-left: 4px;

    &:hover {
      text-decoration: underline;
    }
  }
}
</style>
```

十二、Markdown 渲染工具封装

12.1 Markdown 渲染器

创建 `src/utils/markdown.js`：

```
/**
 * Markdown 渲染工具
 * 用于 Day 11 智能体对话中 AI 回复的 Markdown 渲染
 */
```

```
import MarkdownIt from 'markdown-it'

// 创建 markdown-it 实例, 启用 HTML 支持
const md = new MarkdownIt({
  html: false,          // 禁用 HTML 标签 (安全考虑)
  linkify: true,        // 自动将 URL 转为链接
  typographer: true,    // 启用排版优化 (如引号替换)
  breaks: true,         // 将 \n 转为 <br>
})

/**
 * 将 Markdown 文本渲染为 HTML
 * @param {string} text - Markdown 文本
 * @returns {string} 渲染后的 HTML 字符串
 */
export function renderMarkdown(text) {
  if (!text) return ''
  return md.render(text)
}

export default md
```

十三、入口文件整合

13.1 修改 main.js

修改 `src/main.js`:

```
/**
 * 应用入口文件
 * - 创建 Vue 应用实例
 * - 注册全局插件 (Element Plus、Router、Pinia)
 * - 挂载应用
 */
import { createApp } from 'vue'

// 全局样式
import '@assets/styles/global.scss'

// Element Plus
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import zhCn from 'element-plus/es/locale/lang/zh-cn'

// 核心模块
import App from './App.vue'
import router from './router'
import pinia from './stores'

// — 创建并配置应用 —
```

```
const app = createApp(App)

// 注册插件
app.use(pinia) // 状态管理
app.use(router) // 路由
app.use(ElementPlus, { locale: zhCn }) // UI 组件库 (中文语言包)

// 挂载到 DOM
app.mount('#app')
```

13.2 修改 App.vue

修改 `src/App.vue`：

```
<template>
  <router-view />
</template>

<script setup>
// 根组件仅作为路由出口
</script>

<style>
/* 全局样式已在 global.scss 中定义 */
</style>
```

13.3 修改 index.html

修改 `frontend/index.html`：

```
<!DOCTYPE html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>RSOD Agent Platform</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

13.4 创建页面占位组件

为路由中引用的页面创建占位组件，后续各 Day 完善。

创建 `src/views/ChatPage.vue`：

```
<template>
  <div class="page-container">
    <h2>智能对话</h2>
    <p>Day 11 实现 LangGraph 多 Agent 对话功能</p>
  </div>
</template>
```

创建 `src/views/DetectionPage.vue` :

```
<template>
  <div class="page-container">
    <h2>检测工作台</h2>
    <p>Day 8 实现单图/批量/视频/摄像头检测功能</p>
  </div>
</template>
```

创建 `src/views/TrainingPage.vue` :

```
<template>
  <div class="page-container">
    <h2>模型训练</h2>
    <p>Day 6 实现 YOLOv11 模型训练与监控</p>
  </div>
</template>
```

创建 `src/views/HistoryPage.vue` :

```
<template>
  <div class="page-container">
    <h2>历史记录</h2>
    <p>Day 10 实现检测历史记录与统计看板</p>
  </div>
</template>
```

创建 `src/views/DashboardPage.vue` :

```
<template>
  <div class="page-container">
    <h2>数据看板</h2>
    <p>Day 10 实现 ECharts 数据可视化看板</p>
  </div>
</template>
```

13.5 清理默认文件

删除 Vite 脚手架生成的默认文件：

```
# 删除默认组件和样式
rm src/components/HelloWorld.vue
rm src/style.css
rm src/assets/vite.svg
rm src/assets/vue.svg
```

十四、验证与测试

14.1 启动前端开发服务器

```
cd frontend
npm run dev
```

预期输出：

```
VITE v8.x.x  ready in xxx ms

→ Local:   http://localhost:5173/
→ Network: http://192.168.x.x:5173/
→ press h + enter to show help
```

浏览器自动打开 `http://localhost:5173`，应看到登录页面（因为路由守卫检测到未登录，自动跳转到 `/login`）。

14.2 验证清单

验证项	操作步骤	预期结果
前端启动	<code>npm run dev</code>	无报错，浏览器打开登录页
登录页面	访问 <code>/login</code>	显示登录表单，背景渐变色
注册页面	点击"立即注册"	跳转到注册页面
表单验证	提交空表单	显示验证错误提示
注册流程	填写注册表单并提交	注册成功后跳转到登录页
登录流程	使用注册的账号登录	登录成功，跳转到智能对话页
侧边栏导航	点击各菜单项	页面切换正常
路由守卫	退出登录后直接访问 <code>/detection</code>	自动跳转到登录页
退出登录	点击退出按钮	清除 Token，跳转登录页

14.3 测试登录流程

1. 确保后端服务正在运行：

```
cd backend
python main.py
```

2. 确保 Docker 基础设施已启动：

```
docker compose up -d
```

3. 打开前端 `http://localhost:5173`，使用之前注册的账号登录

4. 打开浏览器开发者工具 → Network 面板，观察：

- `/api/auth/login` 请求是否正常
- 请求头中是否包含 `Authorization: Bearer xxx`
- 登录成功后侧边栏是否正常显示

十五、常见问题排查

15.1 依赖安装失败

```
# 清除缓存重新安装
rm -rf node_modules package-lock.json
npm install
```

15.2 SCSS 变量无法使用

确保 `vite.config.js` 中正确配置了 `additionalData`：

```
css: {
  preprocessorOptions: {
    scss: {
      additionalData: `@use "@assets/styles/variables.scss" as *;`,
    },
  },
},
```

15.3 代理不生效（404 或跨域错误）

1. 检查后端是否启动：`http://localhost:8000/docs`
2. 检查 `vite.config.js` 中 proxy 配置：target 是否为 `http://localhost:8000`
3. 重启前端开发服务器

15.4 Element Plus 中文不生效

确保 `main.js` 中传入了 `zhCn` 语言包：

```
import zhCn from 'element-plus/es/locale/lang/zh-cn'
app.use(ElementPlus, { locale: zhCn })
```

15.5 路由跳转白屏

1. 检查 `router/index.js` 中组件路径是否正确
2. 检查占位组件是否已创建
3. 打开浏览器控制台查看错误信息

15.6 Pinia store 在路由守卫中无法使用

路由守卫执行时 Pinia 可能尚未初始化。解决方案：在路由守卫中直接使用 `localStorage` 检查 Token，而非通过 store。

15.7 登录后刷新页面状态丢失

当前实现中 Token 和用户信息已持久化到 `localStorage`，刷新页面后 Pinia store 会自动从 `localStorage` 恢复数据，不会出现状态丢失。

附录 A：文件创建清单

文件路径	操作	说明
frontend/.env	新建	环境变量配置
frontend/.env.example	新建	环境变量模板
frontend/vite.config.js	修改	添加路径别名、SCSS、代理
frontend/index.html	修改	设置标题和语言
frontend/src/main.js	修改	注册插件
frontend/src/App.vue	修改	简化为路由出口
frontend/src/assets/styles/variables.scss	新建	SCSS 变量
frontend/src/assets/styles/reset.scss	新建	样式重置
frontend/src/assets/styles/global.scss	新建	全局样式
frontend/src/utils/request.js	新建	Axios 封装
frontend/src/utils/stream.js	新建	SSE 流式处理
frontend/src/utils/markdown.js	新建	Markdown 渲染
frontend/src/stores/index.js	新建	Pinia 入口
frontend/src/stores/user.js	新建	用户状态
frontend/src/stores/agent.js	新建	智能体状态
frontend/src/router/index.js	新建	路由配置 + 守卫
frontend/src/api/auth.js	新建	认证 API
frontend/src/components/layout/AppHeader.vue	新建	顶部导航栏
frontend/src/components/layout/AppSidebar.vue	新建	侧边栏菜单
frontend/src/components/layout/MainLayout.vue	新建	主布局
frontend/src/views/LoginPage.vue	新建	登录页
frontend/src/views/RegisterPage.vue	新建	注册页
frontend/src/views/ChatPage.vue	新建	对话页（占位）
frontend/src/views/DetectionPage.vue	新建	检测页（占位）
frontend/src/views/TrainingPage.vue	新建	训练页（占位）
frontend/src/views/HistoryPage.vue	新建	历史页（占位）
frontend/src/views/DashboardPage.vue	新建	看板页（占位）

附录 B：依赖安装命令汇总

```
cd frontend

# 生产依赖
npm install element-plus @element-plus/icons-vue vue-router@4 pinia axios markdown-it

# 开发依赖
npm install -D sass
```

附录 C：与后端 API 对照表

前端 API 函数	后端路由	方法	说明
registerApi()	/api/auth/register	POST	用户注册
loginApi()	/api/auth/login	POST	用户登录
getUserInfoApi()	/api/auth/me	GET	获取当前用户